

# Documentacion del Proyecto: Simulación de Cafetería con Hilos en JavaFX

Autor: Joel Novo Pampillón

29 de octubre de 2025

## Introducción del Proyecto

En este proyecto desarrollaremos una aplicacion la cual simulara una cafeteria. El principal objetivo es que con Thread(hilos) podamos gestionar multiples pedidos de clientes de manera simultanea, optimizando el tiempo de espera y mejorando la eficiencia del servicio. Para esto hemos utilizado JavaFx

## 1. Arquitectura del Sistema

### 1.1. Modelo

#### Clases y responsabilidades

- **Cliente**
  - Representa a un cliente con nombre y tiempo de espera.
  - Implementa `Runnable` → cada cliente corre en su propio hilo.
  - Pide un café y espera hasta que se lo entreguen o se marche.
- **Camarero**
  - Extiende `Thread`.
  - Atiende a los clientes en orden.
  - Simula el tiempo de preparación con `Thread.sleep()`.
- **Cafeteria**
  - Gestiona la cola de clientes (`Queue`).
  - Sincroniza la interacción entre camareros y clientes.
  - Controla el estado (abierta o cerrada).
  - Comunica eventos al controlador (interfaz).

### Conceptos de programación concurrente:

- Uso de `synchronized`, `wait()` y `notifyAll()` para coordinar hilos.
- Evita condiciones de carrera.
- Controla cuándo los clientes se van si esperan demasiado.

### 1.2. Vista (`cafeteria.fxml`)

Define la interfaz en FXML

- Un `TextArea` para mostrar eventos de la simulación.
- Un `Button` para iniciar la simulación.

### Ventajas:

- Separa la parte visual del código Java.
- Facilita cambiar el diseño sin tocar la lógica.

### 1.3. Controlador (`ControladorCafeteria.java`)

Conecta la vista con el modelo:

- Inicializa la clase `Cafeteria`.
- Responde al clic del botón *Iniciar simulación*.
- Crea y lanza los hilos (`Camarero`, `Cliente`).
- Actualiza la interfaz con los eventos mediante `Platform.runLater`.

Esto permite que la interfaz siga funcionando aunque la simulación corra en segundo plano.

### 1.4. Launcher (`App.java`)

- Carga el archivo FXML desde los recursos.
- Muestra la ventana principal.
- Configura el título y el tamaño.

## 2. Casos de Uso

| ID  | Caso de Uso           | Actor    | Flujo Principal                                   |
|-----|-----------------------|----------|---------------------------------------------------|
| CU1 | Entrar a la cafetería | Cliente  | Llega, se agrega a la cola y espera café          |
| CU2 | Preparar café         | Camarero | Obtiene cliente, prepara café, entrega y notifica |
| CU3 | Recibir café          | Cliente  | Despierta, recibe café y muestra mensaje          |
| CU4 | Cliente se va         | Cliente  | Supera tiempo de espera y se retira               |
| CU5 | Iniciar simulación    | Usuario  | Se crean hilos de clientes y camareros            |
| CU6 | Cierre de cafetería   | Sistema  | Cafetería se cierra y camareros finalizan         |
|     |                       |          |                                                   |

### 3. Interfaz

La interfaz gráfica del proyecto fue desarrollada utilizando **JavaFX** y definida mediante el archivo `cafeteria.fxml`. Su diseño busca representar de forma visual y dinámica el funcionamiento interno de la simulación, permitiendo observar los eventos que ocurren en tiempo real.

#### Elementos principales de la interfaz

- **Título de la ventana:** Muestra el nombre del proyecto “*Cafeteria Novotoast*”, proporcionando un contexto claro al usuario.
- **Área de logs (`TextArea`):**
  - Presenta los eventos de la simulación en tiempo real, como la llegada de clientes, la preparación de cafés y los clientes que se van.
  - Cada línea representa una acción o cambio de estado dentro de la cafetería.
- **Contadores de estado:**
  - **Label clientes contentos:** Muestra la cantidad de clientes que recibieron su café correctamente.
  - **Label clientes enfadados:** Indica el número de clientes que abandonaron la cafetería por espera.
  - Ambos contadores se actualizan automáticamente conforme avanza la simulación.
- **Botón de inicio (`Button Iniciar Simulación`):**
  - Inicia el proceso de simulación al ser presionado.
  - Desactiva temporalmente el botón mientras la simulación está en ejecución para evitar reinicios múltiples.

#### Diseño y organización

La disposición de los elementos se estructura de forma vertical para una lectura intuitiva:

- En la parte superior: el título principal.
- Justo debajo: los contadores de clientes contentos y enfadados.
- En el centro: el área de logs.
- Al final: el botón para iniciar la simulación.

Este diseño busca simplicidad y funcionalidad, centrando la atención del usuario en los eventos del sistema.

### 4. Conclusiones

- Se implementó una simulación de cafetería con multihilos y JavaFX.
- La coordinación de clientes y camareros se logró con `wait()` y `notifyAll()`.